Chapter 12 — Fine-Tuning & Open Models

12.1 When to Fine-Tune (and When Not To)

Conceptual Deep Dive

Fine-tuning changes model *behaviour* — format, tone, domain jargon, task skill, tool-use style, latency/cost via distilling a large model into a small one. It is a poor way to add *knowledge* (use RAG) and can cause forgetting. Decision ladder: better prompt → few-shot → RAG → fine-tune a small model → continued pretraining (rare). Fine-tune when: you have ≥ hundreds of high-quality examples, a prompt can't hit the quality bar, you need lower cost/latency at volume, or you need on-prem/privacy. Open-weight families to know: Llama, Qwen, Mistral/Mixtral, Gemma, DeepSeek, Phi. Check licences (commercial use, attribution, usage restrictions).

Cross-Connection Scenarios

Evals decide the need (10); data from logs (14); serving the result (13); distillation reduces cost (2.2).

Production Anti-Patterns

- Fine-tuning to inject facts that change weekly.
- Skipping a prompt-engineering baseline.
- Ignoring licence terms.

Minimal Working Code

```
def should_finetune(prompt_score: float, rag_score: float, target: flo
at,
                    n_examples: int, knowledge_changes_often: bool) -> str:
    if max(prompt_score, rag_score) >= target:
        return "no: current approach meets target"
    if knowledge_changes_often:
        return "no: improve retrieval instead"
    if n_examples < 200:
        return "collect more data first"
    return "yes: try LoRA SFT on a small open model"

print(should_finetune(0.70, 0.78, 0.9, 1500, False))
```

12.2 Dataset Curation

Conceptual Deep Dive

Data quality dominates quantity (LIMA: ~1,000 curated examples can align style). Pipeline: collect (logs, SME-written, synthetic via a stronger model — check provider terms) → clean (dedupe exact + near-duplicate with MinHash, strip PII, fix encoding) → filter (heuristics, classifier or LLM-judge quality scores) → format into **chat templates** (the exact template used at inference, incl. special tokens) → mask loss to **assistant tokens only** → split train/val/test without leakage → inspect samples by hand. For preference tuning, build (prompt, chosen, rejected) triples from human rankings, judge-ranked samples, or on-policy generations. Balance task mix; add refusal and "I don't know" examples; keep a general-capability replay set to limit forgetting.

Cross-Connection Scenarios

PII scrubbing (14); judge filters (10.3); deduplication/embedding clustering (8).

Production Anti-Patterns

- Training on model-generated data with errors unreviewed (error amplification).
- Template mismatch between training and serving.
- Test-set leakage through near-duplicates.

Minimal Working Code

```
import hashlib
import re

def normalize(t: str) -> str:
    return re.sub(r"\s+", " ", t.lower()).strip()

def dedupe(rows):
    seen, out = set(), []
    for r in rows:
        h = hashlib.md5(normalize(r["prompt"] + r["response"]).encode()).hexdigest()
        if h not in seen:
            seen.add(h)
            out.append(r)
    return out

def to_chat(row, system="You are a helpful assistant."):
    return {"messages": [{"role": "system", "content": system},
                         {"role": "user", "content": row["prompt"
]},
                         {"role": "assistant", "content": row["respons
e"]}]}

rows = dedupe([{"prompt": "Hi", "response": "Hello"}, {"prom
pt": "hi ", "response": "hello"}])
print(len(rows), to_chat(rows[0]))
```

12.3 Supervised Fine-Tuning and Parameter-Efficient Methods (LoRA, QLoRA)

Conceptual Deep Dive

SFT minimises cross-entropy on target responses. Full fine-tuning updates all weights (≈16 bytes/param with Adam — costly). **PEFT** trains a tiny set of extra parameters. **LoRA** freezes W and learns a low-rank update: W' = W + (α/r)·B·A, with A∈ R^{r×k}, B∈ R^{d×r}, rank r typically 8–64; trainable params drop by ~100–1000×, and adapters can be swapped or merged. Targets: all linear layers (q,k,v,o,gate,up,down) usually beats attention-only. **QLoRA**: quantise the frozen base to 4-bit NF4 (with double quantisation and paged optimisers) and train LoRA adapters in bf16 — fine-tune a 70B model on a single large GPU. Variants: DoRA, rsLoRA, LoRA+, adapter merging. Hyperparameters: LR 1e-4–2e-4 for LoRA (lower, ~1e-5, for full FT), 1–3 epochs, warmup, cosine decay, effective batch via gradient accumulation, packing for short samples. **Unsloth** accelerates LoRA/QLoRA with custom kernels and lower VRAM; alternatives: Axolotl, TRL, LLaMA-Factory, torchtune.

Cross-Connection Scenarios

Quantisation (12.5); hardware math (5.6); evaluation of the tuned model (10); multi-LoRA serving (13).

Production Anti-Patterns

- Too many epochs → memorisation and capability loss.
- Computing loss over the prompt tokens.
- Merging a LoRA into a quantised base without care (quality loss).

Minimal Working Code

```
# pip install transformers peft trl bitsandbytes datasets
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer
from datasets import load_dataset

BASE = "Qwen/Qwen2.5-1.5B-Instruct"
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_use_double_quant=True)
model = AutoModelForCausalLM.from_pretrained(BASE, quantization_config=bnb, device_map="auto")
tok = AutoTokenizer.from_pretrained(BASE)

lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05, task_type="CAUSAL_LM",
                  target_modules=["q_proj", "k_proj", "v_proj", "o_pro
j",
                                  "gate_proj", "up_proj", "down_proj"])
ds = load_dataset("json", data_files="train.jsonl", split="train")   # {&qu
ot;messages":[...]}

trainer = SFTTrainer(
    model=model, train_dataset=ds, peft_config=lora, processing_class=tok,
    args=SFTConfig(output_dir="out", per_device_train_batch_size=2, gradient_accumulation_steps
=8,
                   learning_rate=2e-4, num_train_epochs=2, lr_scheduler_type="cosine",
                   warmup_ratio=0.03, bf16=True, logging_steps=10, assistant_only_loss=True),
)
trainer.train()
trainer.save_model("out/adapter")

# Unsloth variant:  from unsloth import FastLanguageModel
#   model, tok = FastLanguageModel.from_pretrained("unsloth/Llama-3.2-3B", load_in_4bit=True)
#   model = FastLanguageModel.get_peft_model(model, r=16, lora_alpha=16)
```

12.4 Preference Optimisation: RLHF, DPO, ORPO

Conceptual Deep Dive

RLHF: train a reward model on human preference pairs, then optimise the policy with PPO under a KL penalty to the reference model — powerful but complex and unstable. **DPO** removes the reward model and RL loop: with pairs (x, y_w, y_l) it minimises -log σ(β[log π(y_w|x)/π_ref(y_w|x) – log π(y_l|x)/π_ref(y_l|x)]), directly increasing the relative likelihood of chosen over rejected responses; β (0.1–0.5) controls deviation from the reference. **ORPO** folds an odds-ratio preference penalty into SFT, needing no reference model and a single stage; **SimPO**, **KTO** (binary good/bad labels), **IPO** are related. **GRPO** and RL with verifiable rewards (math/code) train reasoning behaviour using group-relative advantages without a value model. Order: SFT first, then preference tuning. Watch for length exploitation, reward hacking, and likelihood displacement.

Cross-Connection Scenarios

Preference data from judge/eval pipelines (10); safety alignment (14); reasoning models (5.5).

Production Anti-Patterns

- DPO on noisy preference labels (garbage in, sycophancy out).
- Skipping SFT and applying DPO to a base model.
- Optimising a proxy reward until quality collapses.

Minimal Working Code

```
import math

def dpo_loss(lp_w, lp_l, ref_lp_w, ref_lp_l, beta=0.1) -> float:
```

```
    """Inputs are summed log-probs of chosen (w) and rejected (l) under policy and referen
ce."""
    margin = beta * ((lp_w - ref_lp_w) - (lp_l - ref_lp_l))
    return -math.log(1 / (1 + math.exp(-margin)))

print(dpo_loss(-10, -14, -11, -12))

# TRL:  from trl import DPOTrainer, DPOConfig
#   DPOTrainer(model, ref_model=None, args=DPOConfig(beta=0.1), train_dataset=ds_with_prompt_chosen_rejec
ted)
# ORPO: from trl import ORPOTrainer, ORPOConfig
```

12.5 Quantization: GPTQ, AWQ, GGUF, bitsandbytes

Conceptual Deep Dive

Quantisation stores weights in fewer bits: w_q = round(w/s) + z, dequantised on the fly. **Post-training** methods: **GPTQ** (layer-wise, uses second-order Hessian info to compensate rounding errors; 4-bit/3-bit, GPU), **AWQ** (activation-aware: protects the ~1% salient channels by scaling; fast, strong at 4-bit), **GGUF** (llama.cpp file format with k-quants such as Q4_K_M, Q5_K_M, Q8_0; CPU/Metal/GPU hybrid, ideal for local use), **bitsandbytes** (NF4/int8 on-the-fly, used for QLoRA), **FP8** (H100+ native), **HQQ**, **EXL2**. Quantisation-aware training exists but PTQ dominates. Guidance: Q8 ≈ lossless, Q5/Q6 near-lossless, 4-bit small loss (bigger on small models and reasoning/code/multilingual), 3-bit and below noticeably degrades. KV-cache quantisation (fp8/int8) saves memory for long contexts. Always re-run your evals on the quantised model.

Cross-Connection Scenarios

Memory math (5.6); serving engines support specific formats (13); edge/local deployment with Ollama (13).

Production Anti-Patterns

- Assuming a leaderboard-perplexity delta predicts your task's delta.
- Quantising a tiny model to 3-bit.
- Mixing quant format and engine incompatible combos.

Minimal Working Code

```
import numpy as np

def quantize_int4_groupwise(w: np.ndarray, group=64):
    w = w.reshape(-1, group)
    scale = np.abs(w).max(axis=1, keepdims=True) / 7
    q = np.clip(np.round(w / scale), -8, 7).astype(np.int8)
    return q, scale

def dequantize(q, scale):
    return q * scale

w = np.random.default_rng(0).standard_normal(4096).astype(np.float32)
q, s = quantize_int4_groupwise(w)
err = np.abs(dequantize(q, s).ravel() - w).mean()
print(f"mean abs error {err:.4f}")

# AutoAWQ:   AutoAWQForCausalLM.from_pretrained(path).quantize(tok, quant_config={"w_bit":4,&qu
ot;q_group_size":128,"zero_point":True})
# GGUF:      python convert_hf_to_gguf.py model_dir --outtype f16 ; llama-quantize model.gguf model-Q4_K_
M.gguf Q4_K_M
```

12.6 Hosting, Merging and Distillation of Open Models

Conceptual Deep Dive

Distribution: Hugging Face Hub (model cards, safetensors format — avoid pickle). Merge LoRA adapters into base (merge_and_unload) for single-artifact serving, or keep separate for **multi-LoRA serving** (many tenants on one base). **Model merging** (SLERP, TIES, DARE, model soup) blends capabilities without training. **Distillation**: train a small student on a large teacher's outputs (sequence-level) or logits (KL), often giving the best cost/quality win. **Continued pretraining** on domain corpora helps rare domains. Reproducibility: pin base revision hash, dataset version, seed, library versions, and log hyperparameters.

Cross-Connection Scenarios

Registry and CI (15); serving (13); supply-chain security of weights (14).

Production Anti-Patterns

- Loading untrusted pickle checkpoints (torch.load) — arbitrary code execution.
- No model card documenting data, limitations, evals.
- Shipping adapters without recording the exact base revision.

Minimal Working Code

```
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

def merge_and_save(base_id: str, adapter_dir: str, out_dir: str, revision: str | None = None):
    base = AutoModelForCausalLM.from_pretrained(base_id, torch_dtype="auto", revision=revision)
    merged = PeftModel.from_pretrained(base, adapter_dir).merge_and_unload()
    merged.save_pretrained(out_dir, safe_serialization=True)
    AutoTokenizer.from_pretrained(base_id, revision=revision).save_pretrained(out_dir)
```

12.7 Training Stability, Hyperparameters and Debugging

Conceptual Deep Dive

Most fine-tuning failures are data or configuration, not algorithms. Diagnostics: **loss curve** (should fall smoothly; spikes suggest too-high LR or bad batches; flat suggests masked labels or LR too low), **grad norm** (clip at about 1.0), **eval loss vs train loss** (divergence = overfitting), **sample generations** at checkpoints (the best signal), and **task evals** every N steps. Key knobs: learning rate (LoRA 1e-4 to 2e-4), LoRA rank/alpha (alpha about 2x rank is a common start), epochs (1–3), effective batch size 16–128, max sequence length matching real data, packing, and label smoothing rarely. Check the **chat template** and EOS token: a model that never stops, or stops immediately, usually has a template/EOS bug. Edge cases: padding side errors, loss computed on padding tokens, truncation cutting the assistant answer, bf16 vs fp16 overflow, and catastrophic forgetting visible as drops on general benchmarks.

Cross-Connection Scenarios

Data curation (12.2), evaluation (10), quantised training (12.3), reproducibility (15.4).

Production Anti-Patterns

- Judging by train loss alone.
- Changing five hyperparameters between runs.
- Not saving intermediate checkpoints, so overfitting cannot be rolled back.

Minimal Working Code

```
def diagnose(train_loss: list[float], eval_loss: list[float], grad_nor
m: list[float]) -> list[str]:
    msgs = []
    if max(grad_norm) > 10:
```

```
        msgs.append("gradient spikes: lower LR, enable clipping, inspect batches")
    if len(train_loss) > 10 and abs(train_loss[-1] - train_loss[0]) < 0.02:
        msgs.append("loss flat: check label masking, LR too low, or data not reaching model")
    if eval_loss and eval_loss[-1] > min(eval_loss) * 1.05:
        msgs.append("eval loss rising: overfitting; use earlier checkpoint / fewer epochs")
    return msgs or ["looks healthy"]

print(diagnose([2.0] * 12, [1.8, 1.6, 1.7], [0.5] * 12))
```

```
def check_labels(tokenizer, example: dict, ignore_index: int = -100) -
> dict:
    """Verify loss covers only assistant tokens and the answer ends with EOS.""&
quot;
    ids, labels = example["input_ids"], example["labels"]
    trained = [i for i, l in zip(ids, labels) if l != ignore_index]
    return {"trained_tokens": len(trained), "total": len(ids),
            "ends_with_eos": bool(trained) and trained[-1] == tokenizer.eos_token_id}
```

12.8 Evaluating, Aligning and Safety-Testing Fine-Tuned Models

Conceptual Deep Dive

Evaluate against three baselines: the base model with a good prompt, the previous production model, and (if relevant) the API model you hope to replace. Measure task metrics, **format adherence**, refusal behaviour, hallucination rate, latency, and **regressions on general capability** (a small held-out general set) to catch forgetting. Safety: fine-tuning can erode safety behaviour even on benign data, so re-run red-team and refusal suites after every tune. Release process: model card (data, intended use, limits, eval results), licence check, staged rollout (shadow, canary), and rollback to the prior adapter. Edge cases: eval contamination via synthetic data derived from test inputs, memorised PII from training data (test with extraction probes), and benchmark gains that vanish on out-of-distribution inputs.

Cross-Connection Scenarios

Evals (10), guardrails and red-teaming (14), CI gates and registry (15), serving adapters (13).

Production Anti-Patterns

- Shipping after seeing a good aggregate score without slice checks.
- No general-capability regression set.
- Skipping safety re-tests because "the data was harmless".

Minimal Working Code

```
def release_gate(results: dict, baselines: dict, min_gain=0.02, max_ge
neral_drop=0.03, safety_min=0.98):
    problems = []
    if results["task"] < baselines["prompt"] + min_gain:
        problems.append("does not beat prompt baseline by margin")
    if results["general"] < baselines["general"] - max_general_drop:
        problems.append("general capability regression")
    if results["safety_refusal"] < safety_min:
        problems.append("safety refusal rate below threshold")
    return problems or ["ship to canary"]

print(release_gate({"task": 0.91, "general": 0.70, "safety_refusal": 0.99},
                   {"prompt": 0.86, "general": 0.72}))
```

```
import re

def pii_extraction_probe(generate, known_pii: list[str], prefixes: list[str]) -> list[str]:
```

```
    """Try to make the model emit training-set PII; returns leaked strings.""&qu
ot;
    leaked = []
    for p in prefixes:
        out = generate(p)
        leaked += [s for s in known_pii if re.search(re.escape(s), out)]
    return leaked
```